

North vs South India — Regional Classification

Developer plan for the 18:30 deliverable · Zomato Delivery dataset · Team Hmmmmmmmmmm

Mission. Classify each Zomato order as originating in **South India** (Kerala, Tamil Nadu, Andhra Pradesh, Telangana, Karnataka) or **North India** (every other state), and surface the behavioural patterns that distinguish the two. We have **22 city codes** hidden in `Delivery_person_ID` giving us free ground-truth labels — no reverse geocoding needed. Target: a working classifier, a five-finding insights report, and a 10-slide pitch deck, all on GitHub by **18:30**.

1. The data and how we label it

The 'new' CSV in Downloads/archive (2)/ is byte-identical to our existing data/raw/Zomato Dataset.csv — same 45,584 rows × 20 columns, same schema. No additional pivot signal arrived; instead, the *hidden* regional signal was always inside `Delivery_person_ID`.

1.1 City code extraction

Each rider ID follows the pattern {CITY}RES{##}DEL{##}. A simple regex extracts the city prefix and gives us 22 unique cities.

```
df['city_code'] = df['Delivery_person_ID'].str.extract(r'^([A-Z]+)RES', expand=False)
```

1.2 City → state → region mapping

Code	City	State	Region	≈ Orders
BANG	Bangalore	Karnataka	South	3,193
HYD	Hyderabad	Telangana	South	3,179
MYS	Mysore	Karnataka	South	3,170
COIMB	Coimbatore	Tamil Nadu	South	3,169
CHEN	Chennai	Tamil Nadu	South	3,144
KOC	Kochi	Kerala	South	701
JAP	Jaipur	Rajasthan	North	3,443
RANCHI	Ranchi	Jharkhand	North	3,228
SUR	Surat	Gujarat	North	3,187
MUM	Mumbai	Maharashtra	North	3,173
VAD	Vadodara	Gujarat	North	3,166
INDO	Indore	Madhya Pradesh	North	3,158
PUNE	Pune	Maharashtra	North	3,132
AGR	Agra	Uttar Pradesh	North	763
LUDH	Ludhiana	Punjab	North	758
KNP	Kanpur	Uttar Pradesh	North	740
ALH	Allahabad	Uttar Pradesh	North	740
DEH	Dehradun	Uttarakhand	North	737

Code	City	State	Region	≈ Orders
GOA	Goa	Goa	North	709
AURG	Aurangabad	Maharashtra	North	703
KOL	Kolkata	West Bengal	North	700
BHP	Bhopal	Madhya Pradesh	North	691

1.3 Class balance and base-rate

Aggregating: **South ≈ 16,556 orders (36.3%) · North ≈ 29,028 orders (63.7%)**. Class imbalance is mild — accuracy alone would be misleading at the 64% no-skill floor, so we will report **ROC-AUC, F1, and balanced accuracy** as primary metrics.

Sanity cross-check. Each Restaurant_latitude / longitude already falls inside the bounding box of its mapped state for the city codes we recognise — so the lat/lon column is a valid *fallback* if a future row has an unknown prefix. We will keep it as a backup, not the primary label source.

2. Two-model strategy (the interesting twist)

Lat/lon is a near-perfect predictor of region — a model with coordinates will hit 99%+ accuracy trivially. That is not the insight. The interesting question is:

Can we tell North apart from South orders WITHOUT looking at where the rider is?

If yes, the dataset encodes regional behavioural patterns. If no, regions look operationally identical and the difference is purely geographic.

So we train **two LightGBM binary classifiers** on the same 5-fold split:

	Model A — GEO	Model B — BEHAVIOUR
Features	Restaurant_lat, Restaurant_lon, Delivery_lat, Delivery_lon, Restaurant_state, Delivery_state	Everything EXCEPT coordinates: weather, traffic, vehicle, prep_time, rating, etc.
Expected ROC-AUC	0.99 (near-perfect)	0.65 – 0.85 (the real number is the insight)
Purpose	Upper bound; sanity check on labels	Reveals which non-geo features carry regional signal
What we report	1 line: 'geo-baseline 0.99X — labels are clean'	Full SHAP analysis + per-feature regional effect-size table

The **gap between Model A and Model B AUC** is the headline number. A 0.99 vs 0.78 split would mean 'we can tell North from South ~80% of the time even with the map taken away' — that is the kind of finding judges remember.

3. The pattern hunt — what we look for

Before training, run a structured EDA contrasting the two regions across every column. Each test produces one row in the findings table.

#	Question	Test	Why it might split N/S
F1	Are delivery times longer in one region?	Mann-Whitney U on Time_taken	Traffic/culture, road quality, city density
F2	Does weather distribution differ?	Chi-square on Weather_conditions	South sees more Stormy/Fog (coastal); North more Sandstorms
F3	Different traffic patterns?	Chi-square on Road_traffic	Metro south (Bangalore/Chennai) is famously Jam-heavy
F4	Multi-delivery norms?	Mann-Whitney on multiple_deliveries	Concurrency varies with city density and policy
F5	Rider age skew?	Mann-Whitney on Delivery_rider_age	Urban labour-market demographics
F6	Rating skew?	Mann-Whitney on Delivery_ratings	Different inflation behaviour by region
F7	Vehicle preferences?	Chi-square on Type_of_vehicle	Scooter dominance in south, motorcycle in north?
F8	Order-type mix?	Chi-square on Type_of_order	Snack/Meal/Drinks/Buffer mix shifts with cuisine culture
F9	Festival impact differs?	Welch t-test on Time_taken Festival (Yes/No) per Region	Dussehra (Yes) vs Navratri/Pongal (south) hit different weeks
F10	Time-of-day rhythm?	KS test on order_hour_distribution	Dinner peak runs later in metro south; lunch rules in north
F11	City-tier mix?	Chi-square on City (Metro/Urban/Rural)	South sample is metro-heavy in our data

4. Implementation pipeline

Eight files, all in the existing repo layout. Nothing new outside the established conventions.

#	File	Produces
1	src/regional.py	extract_city_code(), map_to_region(), helpers
2	notebooks/_run_regional_eda.py	regional_summary.json + fig_reg_01..11 PNGs (per-finding plot)
3	notebooks/_run_regional_models.py	Trains Model A and Model B (5-fold KFold seed=42), saves AUC/F1/CM
4	notebooks/_run_regional_shap.py	fig_reg_shap_beeswarm.png + fig_reg_shap_bar.png on Model B
5	notebooks/07_regional.ipynb	Thin notebook wrapping the runners (judges open this first)
6	reports/regional_findings.json	Per-test stat, p-value, effect size, verdict
7	reports/regional_models.json	AUC/F1/accuracy per fold for both models
8	reports/dev_plan_regional.pdf	THIS file (for the audit trail)

4.1 The model contract (Rulebook §6 compliance)

- **One source of cleaning** — reuse `src/data.py::load_clean`. Don't fork cleaning logic.
- **One feature builder** — reuse `src/features.py` for the engineered numeric/cat features.
- **One split** — `KFold(5, shuffle=True, random_state=42)`, same seed as the ETA work, so judges can verify deltas.
- **Two regressors → two classifiers** — swap `LGBMRegressor` for `LGBMClassifier`, `objective='binary'`, `metric='auc'`.
- **No hard-coded results** — every number in the deck/PDF reads from JSON written by the runners.
- **Reproducibility** — model files saved as binary-safe .txt (already covered by .gitattributes).

5. The 18:30 pitch deck (regional findings)

New deck: `reports/pitch_regional_630pm.pptx` (+ PDF). 10 slides, navy + teal + gold theme to match the existing decks. Built from the same JSON-driven approach.

#	Slide	Core content
1	Title	Team chip + repo + the single headline AUC number
2	Mission	Region definition (S = KL/TN/AP/TG/KA, N = rest) + class balance
3	Method — labels for free	City-code extraction from <code>Delivery_person_ID</code> , 22-city table
4	Model A — geo baseline	AUC ≈ 0.99, confusion matrix, 'labels are clean' callout
5	Model B — behaviour only	AUC value as the centrepiece, SHAP bar, 5 top features
6	The gap	Model A AUC vs Model B AUC — what survives the geography blackout
7	Pattern 1–3	Three strongest non-geo findings (e.g. weather, traffic, time)
8	Pattern 4–5	Two more (e.g. vehicle mix, order type)
9	Caveats + limitations	Class imbalance, sample size per city, no temporal split
10	Q&A + repo	github.com/TheClazer/... + final commit hash

6. Risk register

Risk	Mitigation
Model B AUC < 0.55 — almost no behavioural signal. Frame slide 6 as 'Behavioural patterns are nearly identical — the difference between South and North is tiny.'	
A city code maps to wrong state	Cross-validate every code by checking the median lat/lon of its rows falls inside the state's bounding box
Hidden 23rd city code we missed	<code>map_to_region()</code> falls back to lat/lon-based label for any unknown prefix; rows with no city code are filtered out
Class imbalance distorts metrics	Report ROC-AUC, balanced accuracy, and per-class F1 — not raw accuracy. Use <code>class_weight='balanced'</code>
Time pressure — 4 hours to 18:30	Models are LightGBM 5-fold (~1 min each). 80% of the time goes to EDA plots + deck. Do NOT touch the model
GitHub push fails at 18:25	Same hotspot/zip-and-email fallback as the existing rulebook risk register.

7. Timeline (4 hours)

Window	Task
14:30 – 14:50 (20m)	src/regional.py + label sanity check via lat/lon bounding box cross-validation
14:50 – 15:30 (40m)	Regional EDA runner — 11 statistical tests + matched plots; write regional_findings.json
15:30 – 16:00 (30m)	Model A (geo) — 5-fold AUC, confusion matrix, save model + metrics
16:00 – 16:40 (40m)	Model B (behaviour) — same 5-fold, save SHAP plots
16:40 – 17:00 (20m)	Two-model comparison plot (AUC bar), feature-effect-size table
17:00 – 17:30 (30m)	Build pitch_regional_630pm.pptx, convert to PDF
17:30 – 17:50 (20m)	README update (regional section), commit, push, tag <code>regional-final</code>
17:50 – 18:20 (30m)	Dry-run + buffer for any last-minute fix
18:20 – 18:30 (10m)	Final upload + chat announcement

8. What we need from you (Rayyan)

- **Sign off on the city-code → state mapping in §1.2.** If any of the 22 codes are wrong, fix here, not later.
- **Confirm the South definition.** You said Kerala, Tamil Nadu, Andhra, Telangana, Karnataka — Goa often gets folded into south coastal; the plan keeps it North per your spec. Say if you want it South.
- **Confirm the pitch substitution.** Should the 18:30 pitch *replace* the existing 18:00 ETA pitch, or be a *second* deck? Plan assumes second deck (pitch_regional_630pm.pdf); the ETA work stays untouched.
- **Confirm metric priority.** Plan reports AUC + F1 + balanced accuracy. Say if you want a different headline metric.

Once you reply with the four answers above (or just say 'looks good, proceed'), the runners and deck are ~3.5 hours of straight work and we make 18:30 comfortably.

Repo: github.com/TheClazer/Zomato-delivery-estimation · **Branch:** p1/pipeline · **Document version:** v1, generated by `notebooks/_build_devplan.py` from live data exploration.